ONLINE BOUTIQUE ON AMAZON EKS

Tài liệu thiết kế kỹ thuật

Hệ thống được xây thế nào và vì sao

Mã tài liệu	OBE-TDD-001-VI
Phiên bản	2.1
Ngày	2026-10-10
Trạng thái	Đã duyệt cho môi trường dev
Người phụ trách	khaipd18 (DevOps / Cloud)
Đối tượng đọc	Kỹ sư Cloud và DevOps, người review
Phân loại	Nội bộ
Repository	https://github.com/khaipd18/online-boutique-gitops-pipeline

Kiểm soát tài liệu

Lịch sử thay đổi

Phiên bản	Ngày	Thay đổi	Người thực hiện
1.0	2026-10-09	Phát hành lần đầu.	khaipd18
1.1	2026-10-10	Chạy thật trên tài khoản mới; Terraform quản lý log group.	khaipd18
1.2	2026-10-10	Triển khai đầy đủ trên EKS, ảnh chụp, access entry, ba node, quyết định D11.	khaipd18
2.0	2026-10-10	Viết lại cho ngắn gọn: mỗi mảng một trang, bỏ phần lặp.	khaipd18
2.1	2026-10-10	GitOps write-back qua pull request khi main có ruleset.	khaipd18

Tài liệu liên quan

- OBE-RUN-001-VI Sổ tay vận hành: cách chạy và xử lý sự cố
- Bản tiếng Anh: `docs/manuals/technical-design.en.pdf` (OBE-TDD-001)
- `docs/infrastructure/`: sơ đồ HLD/LLD sửa được và YAML spec tương ứng

Mục lục

1 Tổng quan 3
2 Hạ tầng AWS 4
2.1 Mạng 4
2.2 EKS, ECR và state 4
2.3 Quyền cho CI (GitHub OIDC) 5
3 Nền tảng Kubernetes 6
4 Pipeline triển khai 8
5 Bảo mật 9
6 Quyết định thiết kế 10
7 Kiểm chứng 11
8 Rủi ro và roadmap 12
Phụ lục. Phiên bản 13

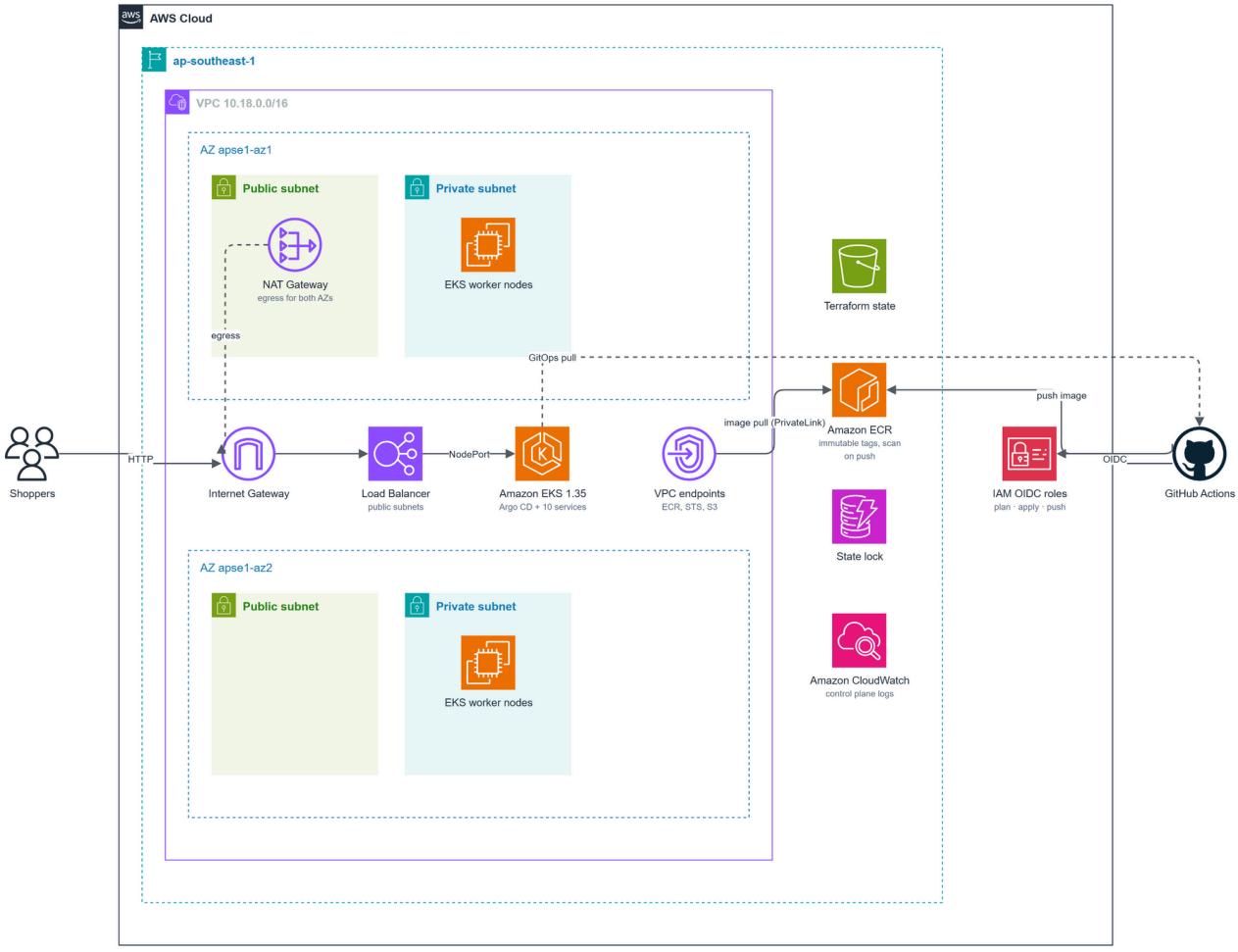
1 Tổng quan

Đây là gì. Online Boutique của Google (10 microservice viết bằng 5 ngôn ngữ, gọi nhau qua gRPC) chạy trên Amazon EKS. Toàn bộ phần bao quanh được xây trong repository này: hạ tầng dạng code, CI/CD, GitOps và các lớp bảo mật. Code ứng dụng trong `src/` là code upstream và được giữ nguyên.

Mục tiêu.

- Không có access key AWS dài hạn ở đâu cả; chỉ code đã review mới thay đổi được AWS.
- Phát hiện cấu hình sai và image có lỗ hổng trước khi deploy.
- Dựng lại được cluster chỉ từ Git; chuyển sang tài khoản AWS khác không cần sửa code.
- Môi trường dev dựng lên và tắt đi rẻ, nhanh.

Hiện trạng. Đã triển khai và kiểm tra end-to-end trên EKS ngày 10/10/2026 (mục 7), sau đó destroy để tiết kiệm chi phí. Có một môi trường (`dev`) ở `ap-southeast-1`.



Hình 1: Kiến trúc tổng thể

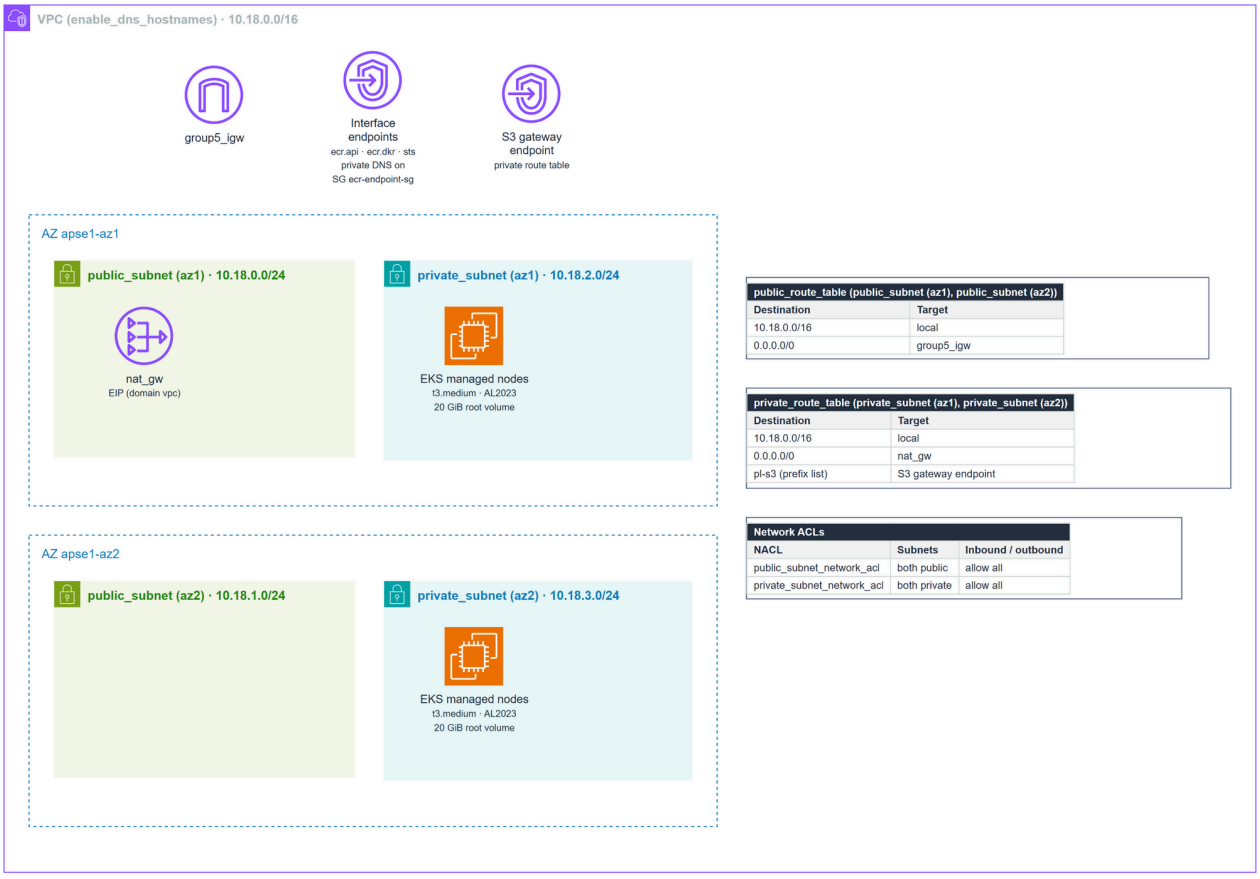
Người dùng vào shop qua load balancer ở public subnet; các worker node nằm trong private subnet. Node truy cập ECR, STS và S3 qua VPC endpoint, ra internet qua một NAT Gateway. Argo CD chạy trong cluster và kéo trạng thái mong muốn từ GitHub về.

2 Hạ tầng AWS

2.1 Mạng

Subnet	AZ ID	CIDR	Chứa
public (az1)	apse1-az1	10.18.0.0/24	NAT Gateway, load balancer
public (az2)	apse1-az2	10.18.1.0/24	load balancer
private (az1)	apse1-az1	10.18.2.0/24	EKS nodes, interface endpoints
private (az2)	apse1-az2	10.18.3.0/24	EKS nodes, interface endpoints

- VPC `10.18.0.0/16` trên hai AZ, đặt theo AZ ID vì tên AZ mỗi tài khoản một khác.
- Một NAT Gateway dùng chung cho hai AZ để tiết kiệm (quyết định D3).
- VPC endpoint cho `ecr.api`, `ecr.dkr`, `sts` và S3, nên việc pull image không ra khỏi VPC [AWS].
- Lọc traffic bằng security group và NetworkPolicy của Kubernetes; network ACL cho qua tất cả.



Hình 2: LLD: chi tiết mạng

2.2 EKS, ECR và state

Thành phần	Thiết kế
EKS cluster	`khaipd18-eks-cluster`, Kubernetes 1.35 (standard support tới 27/03/2027 [AWS]); API endpoint public và private; mọi control plane log đẩy về CloudWatch (lưu 365 ngày)
Node	Managed node group, 3 × `t3.medium` (min 1, max 4), Amazon Linux 2023, chỉ nằm trong private subnet
Add-on	VPC CNI có thực thi NetworkPolicy, CoreDNS, kube-proxy, Metrics Server

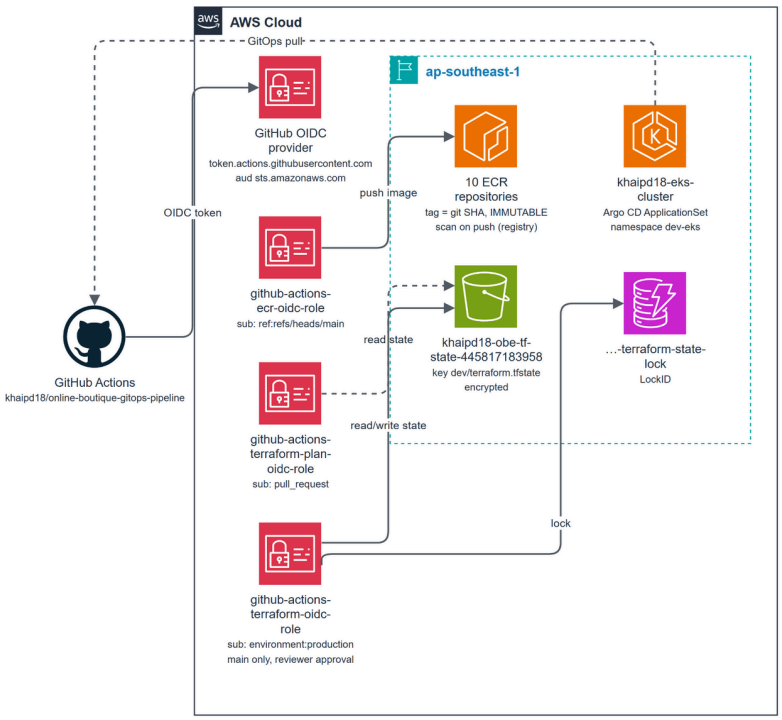
Quyền vào cluster	EKS access entry: người tạo cluster là admin, role console chỉ đọc (`AmazonEKSAdminViewPolicy`) [AWS]
ECR	10 repository, tag immutable (tag = git SHA), scan on push, lifecycle policy dọn image cũ [AWS]
Terraform state	S3 bucket `khaipd18-obe-tf-state-445817183958` (có versioning, mã hóa) và bảng khóa DynamoDB; tạo tay một lần

Terraform chia thành 5 module: `vpc`, `eks`, `ecr`, `vpc-endpoints` và `github-oidc-role`. Account ID không bao giờ được ghi cứng trong code.

2.3 Quyền cho CI (GitHub OIDC)

GitHub Actions nhận credential AWS ngắn hạn từ GitHub OIDC provider. Mỗi role chỉ tin đúng một giá trị `sub` của token [AWS]:

Role	sub được tin	Dùng để
`github-actions-ecr-oidc-role`	`ref:refs/heads/main`	CI push image
`github-actions-terraform-plan-oidc-role`	`pull_request`	`terraform plan` chỉ đọc cho PR
`github-actions-terraform-oidc-role`	`environment:production`	`terraform apply`, chỉ sau khi có người duyệt



GitHub Actions roles			
Role	**Trusted sub (repo:<repo>:…)**	**Permissions**	**Used by**
github-actions-ecr-oidc-role	ref:refs/heads/main	ECR push/pull on the 10 repositories	5 service CI workflows
github-actions-terraform-plan-oidc-role	pull_request	ReadOnlyAccess + state read (plan -lock=false)	terraform.yaml on pull requests
github-actions-terraform-oidc-role	environment:production	AdministratorAccess + state read/write/lock	terraform.yaml apply job, push to main + approval

Hình 3: LLD: CI/CD và IAM OIDC

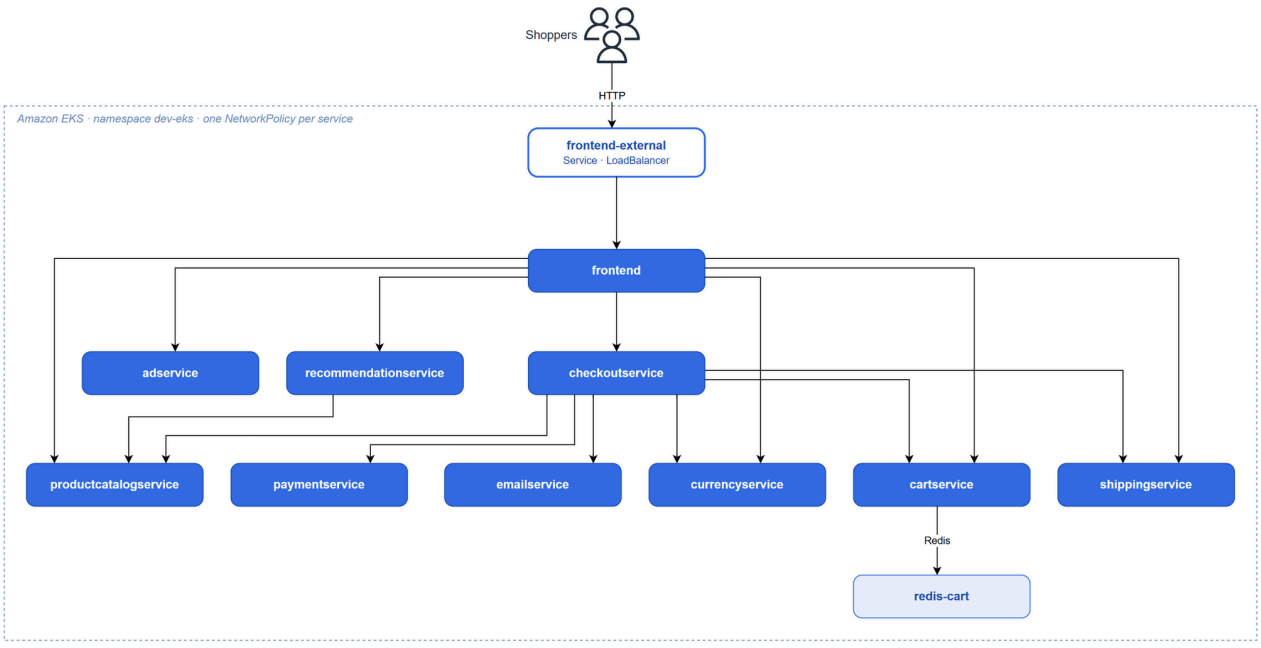
3 Nền tảng Kubernetes

Mọi service chạy trong namespace `dev-eks`. Tất cả release dùng chung một Helm chart (`helm-charts/`); file values của từng service chỉ ghi phần khác biệt.

Service	Ngôn ngữ	Port	Nhận lời gọi từ	CPU req/limit	Memory req/limit
frontend	Go	80 → 8080	any (LoadBalancer)	100m / 200m	64Mi / 256Mi
checkoutservice	Go	5050	frontend	100m / 200m	64Mi / 128Mi
productcatalogservice	Go	3550	frontend, checkout, recommendation	100m / 200m	64Mi / 128Mi
shippingservice	Go	50051	frontend, checkout	100m / 200m	64Mi / 128Mi
cartservice	C#	7070	frontend, checkout	200m / 300m	64Mi / 128Mi
adservice	Java	9555	frontend	200m / 300m	180Mi / 300Mi
currencyservice	Node.js	7000	frontend, checkout	100m / 200m	64Mi / 128Mi
paymentservice	Node.js	50051	checkout	100m / 200m	64Mi / 128Mi
emailservice	Python	8080	checkout	100m / 200m	64Mi / 128Mi
recommendationservice	Python	8080	frontend	100m / 200m	250Mi / 500Mi
redis-cart	Redis	6379	cartservice	100m / 200m	64Mi / 128Mi

An toàn mặc định. Pod chạy bằng user không phải root, file system chỉ đọc, không có Linux capability, không mount service account token. Pod Security Admission (`restricted`) từ chối mọi pod không theo các quy tắc này.

Ai được gọi ai. Mỗi service có một NetworkPolicy chỉ cho các bên gọi được liệt kê trong file values (`allowFrom`) đi vào.

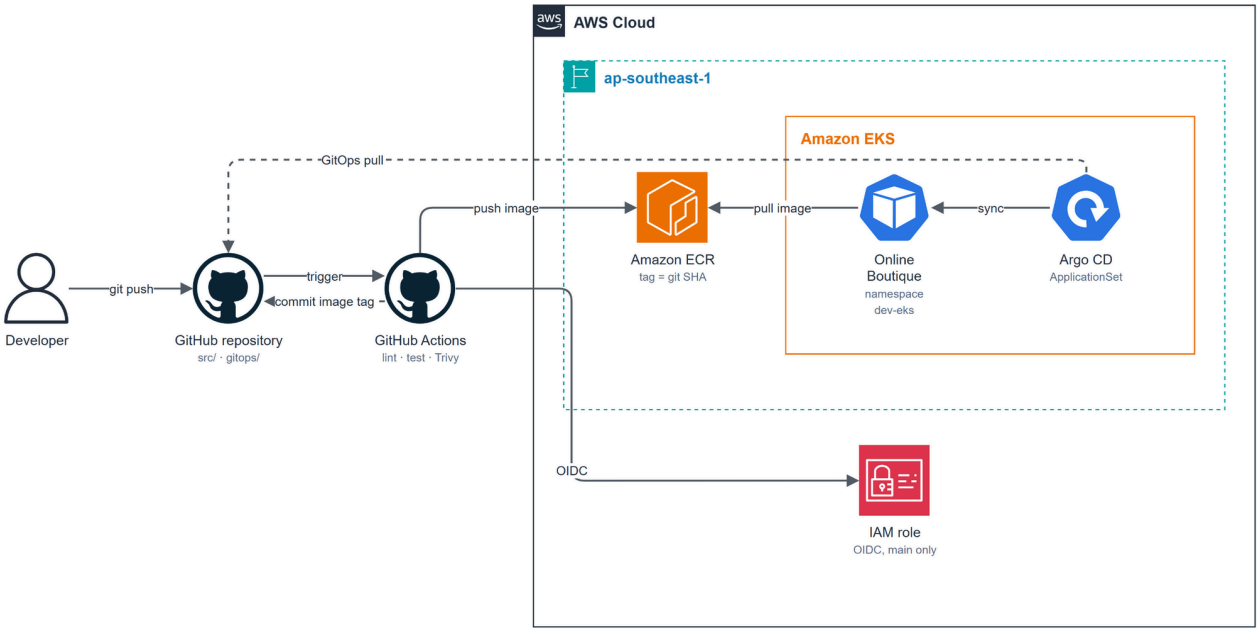


Hình 4: Các lời gọi được phép giữa các service

Khả năng chịu lỗi. `frontend` chạy 2–4 replica với autoscaler theo CPU (HPA), PodDisruptionBudget và rải qua các zone. Các service còn lại chạy một replica. `redis-cart` giữ giỏ hàng trong bộ nhớ nên giỏ hàng mất khi nó khởi động lại.

Monitoring. Prometheus và Grafana (`kube-prometheus-stack`) trong namespace `monitoring`; control plane log trên CloudWatch. Chưa có alert riêng.

4 Pipeline triển khai



Hình 5: Từ commit tới cluster

CI (mỗi ngôn ngữ một workflow). Với mỗi service thay đổi: lint, test và dependency scan → build image → quét bằng Trivy (kết quả lên tab Security của GitHub, lưu SBOM) → push lên ECR với tag là git SHA → ghi tag vào `gitops/dev-eks/` qua một pull request do bot tự mở và tự merge (`main` chỉ nhận pull request). Khi chưa có biến `AWS_ACCOUNT_ID`, CI vẫn lint, test, build, quét và bỏ qua các bước AWS.

Stack	Lint / format	Test	Dependency scan
Go (4 services)	golangci-lint v1.64.8 (blocking)	go test (blocking)	govulncheck v1.8.0 (warning)
.NET (cartservice)	dotnet format (warning)	dotnet test (blocking)	dotnet list package --vulnerable (blocking)
Java (adservice)	google-java-format (warning)	gradle test (blocking)	–
Node.js (2 services)	ESLint 8 (blocking)	–	npm audit (warning)
Python (2 services)	flake8 (blocking)	pytest (blocking when tests exist)	bandit (blocking)

Các bước ghi **warning** cần sửa code upstream mới hết, nên chỉ hiện cảnh báo chứ không làm run fail.

CD. Một Argo CD ApplicationSet tạo 12 Application từ `gitops/dev-eks/` và giữ chúng luôn đồng bộ: thay đổi trong Git được áp dụng, thay đổi tay bị hoàn lại.

Hạ tầng. Pull request sửa `terraform/` sẽ chạy Checkov (chặn nếu lỗi) và `terraform plan` chỉ đọc. Sau khi merge, `terraform apply` chờ người duyệt trong GitHub environment `production`. Nhánh `main` được bảo vệ bằng ruleset `protect-main`.

5 Bảo mật

Rủi ro	Biện pháp
Lộ access key	Không có key: CI dùng GitHub OIDC, VPC CNI dùng IRSA
Thay đổi chưa review tới được AWS	PR chỉ có role chỉ đọc; apply cần `main` và người duyệt
Terraform cấu hình sai	Checkov chặn trước plan và apply
Image có lỗ hổng	Trivy quét trước khi push; finding lên code scanning; mỗi image có SBOM
Image bị ghi đè	Tag ECR immutable
Container bị chiếm quyền	Không chạy root, file system chỉ đọc, không capability, PSA `restricted`
Đi ngang giữa các service	NetworkPolicy cho từng service

Finding đã chấp nhận. 13 finding của Checkov đã được review và giữ trong `terraform/.checkov.baseline`:

Check	Lý do chấp nhận
`CKV_AWS_39` EKS endpoint public	Cần khi không có VPN; sẽ giới hạn sau (roadmap)
`CKV_AWS_58`, `CKV_AWS_158` chưa dùng KMS key riêng	EKS và CloudWatch Logs đã mã hóa dữ liệu bằng key của AWS [AWS]
`CKV_AWS_229–232` NACL mở port	Lọc traffic bằng security group và NetworkPolicy
`CKV2_AWS_11` chưa bật VPC flow log	Chi phí; bật khi lên production
`CKV2_AWS_12` default security group	Không resource nào dùng

6 Quyết định thiết kế

ID	Quyết định	Lý do / khi nào xem lại
D1	GitHub OIDC thay cho access key	Không có gì để lộ hay phải xoay vòng.
D2	Apply phải được duyệt trong GitHub environment	Chỉ merge thôi thì không đổi được hạ tầng.
D3	Một NAT Gateway cho hai AZ	Rẻ hơn cho dev. Lên production thì dùng regional NAT gateway [AWS].
D4	Một Helm chart cho mọi service	Thiết lập bảo mật viết một lần; thêm service chỉ cần một file values.
D5	ApplicationSet với danh sách cố định	Những gì được deploy nhìn thấy rõ trong Git. Xem lại khi có nhiều môi trường.
D6	Tag image = git SHA, immutable	Image nào đang chạy cũng ứng với một commit; rollback = SHA trước đó.
D7	Classic Load Balancer từ một Service	Không cần cài thêm controller cho dev. Xem lại khi cần TLS hay WAF.
D8	IRSA cho VPC CNI	Chạy được ở mọi nơi. EKS Pod Identity giờ đơn giản hơn [AWS].
D9	Khóa state bằng DynamoDB	Hợp với mọi phiên bản Terraform; sau này chuyển sang khóa trên S3.
D10	Lỗ hổng của code upstream để ở mức warning	Code ứng dụng không được bảo trì ở đây.
D11	Argo CD cài trong cluster	Miễn phí và toàn quyền kiểm soát. EKS cũng có thể chạy Argo CD dưới dạng capability được quản lý [AWS].
D12	Mặc định ba node	Toàn bộ hệ thống cần khoảng 37 pod; hai node `t3.medium` không đủ [AWS].

7 Kiểm chứng

Kiểm tra	Kết quả
Triển khai đầy đủ trên EKS (10/10/2026)	74 resource trên 3 node; Argo CD sync đủ 14 app; đặt thử đơn hàng thành công; PSA từ chối pod sai chuẩn và NetworkPolicy chặn 2/2 kết nối không được phép; sau đó đã destroy
Cluster kind (local)	Mọi pod ready; checkout chạy; HPA scale 2 → 3 khi có tải; PDB chặn drain node cuối cùng chạy `frontend`
Terraform	`fmt`, `validate`, Checkov đều pass; plan = 74 resource
CI không có AWS	Cả 5 pipeline theo ngôn ngữ xanh; có kết quả Trivy cho cả 10 image
Lần chạy EKS đầu tiên (05/2026)	Image push qua OIDC và được Argo CD sync (commit của bot `14bc93f`, eb86a9a, 46e66d5)



Hình 6: Kiểm tra bảo mật trên EKS: PSA từ chối pod sai chuẩn; NetworkPolicy chặn các lời gọi không được phép

8 Rủi ro và roadmap

Rủi ro đã biết: một NAT Gateway là điểm lỗi đơn; mười service chỉ có một replica; giỏ hàng mất khi `redis-cart` khởi động lại; vài image có lỗ hổng đã biết; EKS API endpoint mở ra internet (vẫn cần xác thực IAM).

Roadmap, theo thứ tự:

1. Nâng dependency của các service, rồi chuyển các gate lỗ hổng sang chặn.
2. Giới hạn EKS public endpoint; thu hẹp `ecr-endpoint-sg` về `tcp/443`.
3. Regional NAT gateway; AWS Load Balancer Controller có TLS.
4. External Secrets với AWS Secrets Manager; ký image và pin theo digest.
5. Môi trường staging và production; alert theo SLO.
6. Khóa state trên S3; cân nhắc EKS Pod Identity và Argo CD capability.

Phụ lục. Phiên bản

Thành phần	Phiên bản	Khai báo ở
Terraform	1.14.8	terraform.yaml, README
AWS provider	6.39.0	terraform/versions.tf
Kubernetes (EKS)	1.35	terraform/variables.tf
VPC CNI add-on	v1.21.1-eksbuild.7	terraform/variables.tf
CoreDNS add-on	v1.13.2-eksbuild.4	terraform/variables.tf
kube-proxy add-on	v1.35.3-eksbuild.2	terraform/variables.tf
Metrics Server add-on	v0.9.0-eksbuild.11	terraform/variables.tf
kube-prometheus-stack	84.4.0	gitops/argocd/monitoring.yaml
Argo CD	stable manifest	README, step 3
Checkov	3.3.22	.github/workflows/*
Trivy	0.75.0	.github/actions/trivy-scan
Redis (cart store)	8.10.2	gitops/dev-eks/values-redis-cart.yaml